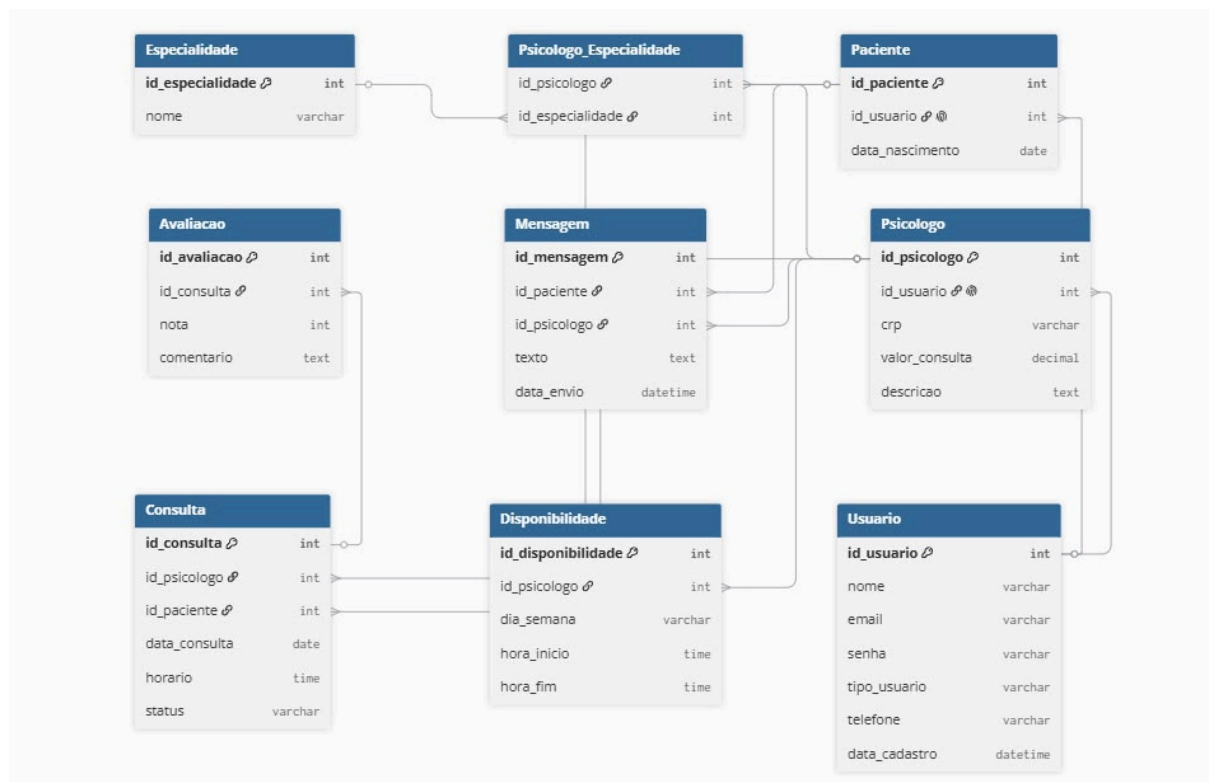


# MODELAGEM DE DADOS — SQL (MySQL)



A modelagem de dados da plataforma foi desenvolvida utilizando o banco de dados relacional MySQL, responsável pelo armazenamento e gerenciamento das informações do sistema. A estrutura foi criada com base nas principais entidades da aplicação, utilizando tabelas relacionadas por meio de chaves primárias (Primary Keys) e chaves estrangeiras (Foreign Keys), garantindo organização, integridade e consistência dos dados.

A tabela **Usuario** representa a entidade principal do sistema, armazenando informações comuns aos usuários da plataforma, como nome, email, senha, telefone, tipo de usuário e data de cadastro. A partir dela são relacionados os perfis específicos de **Paciente** e **Psicólogo**, permitindo separar informações gerais de cadastro dos dados específicos de cada perfil.

A tabela **Paciente** armazena informações dos usuários que buscam atendimento psicológico, enquanto a tabela **Psicólogo** contém informações profissionais, como CRP, valor da consulta e descrição. Ambas possuem relacionamento com a tabela Usuário através da chave estrangeira **id\_usuario**.

A tabela **Especialidade** foi criada para armazenar as áreas de atuação dos psicólogos. O relacionamento entre psicólogos e especialidades é realizado pela tabela intermediária **Psicologo\_Especialidade**, permitindo que um profissional possua várias especialidades e que uma especialidade possa estar associada a diferentes psicólogos.

A tabela **Consulta** representa os agendamentos realizados entre pacientes e psicólogos, armazenando informações como data, horário e status da consulta. A tabela **Avaliação** permite registrar o feedback dos pacientes após os atendimentos, armazenando notas e comentários relacionados às consultas realizadas.

A tabela **Mensagem** foi criada para permitir a comunicação entre pacientes e psicólogos dentro da plataforma, mantendo o registro das mensagens enviadas. Já a tabela **Disponibilidade** armazena os horários disponíveis dos profissionais, auxiliando no processo de agendamento.

Na construção do banco foram utilizados recursos do MySQL, como **AUTO\_INCREMENT** para geração automática de identificadores, **PRIMARY KEY** para identificação única dos registros, **FOREIGN KEY** para criação dos relacionamentos entre tabelas, além das restrições **NOT NULL** e **UNIQUE** para garantir a obrigatoriedade e evitar duplicidade de informações.

Dessa forma, a modelagem desenvolvida atende às funcionalidades principais da plataforma SaaS, permitindo o gerenciamento de usuários, profissionais, consultas, avaliações e comunicação entre pacientes e psicólogos.

## Código

```
CREATE TABLE Usuario (  
  
id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
  
nome VARCHAR(100) NOT NULL,  
  
email VARCHAR(100) NOT NULL UNIQUE,  
  
senha VARCHAR(255) NOT NULL,  
  
tipo_usuario VARCHAR(20) NOT NULL,  
  
telefone VARCHAR(20),  
  
data_cadastro DATETIME DEFAULT CURRENT_TIMESTAMP  
  
);  
  
CREATE TABLE Paciente (  
  
id_paciente INT AUTO_INCREMENT PRIMARY KEY,  
  
id_usuario INT NOT NULL,  
  
data_nascimento DATE,  
  
FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario)
```

);

CREATE TABLE Psicologo (

id\_psicologo INT AUTO\_INCREMENT PRIMARY KEY,

id\_usuario INT NOT NULL,

crp VARCHAR(20) NOT NULL,

valor\_consulta DECIMAL(10,2),

descricao TEXT,

FOREIGN KEY (id\_usuario) REFERENCES Usuario(id\_usuario)

);

CREATE TABLE Especialidade (

id\_especialidade INT AUTO\_INCREMENT PRIMARY KEY,

nome VARCHAR(100) NOT NULL

);

CREATE TABLE Psicologo\_Especialidade (

id\_psicologo INT,

id\_especialidade INT,

PRIMARY KEY (id\_psicologo, id\_especialidade),

FOREIGN KEY (id\_psicologo) REFERENCES Psicologo(id\_psicologo),

FOREIGN KEY (id\_especialidade) REFERENCES Especialidade(id\_especialidade)

);

CREATE TABLE Consulta (

id\_consulta INT AUTO\_INCREMENT PRIMARY KEY,

id\_psicologo INT NOT NULL,

id\_paciente INT NOT NULL,

data\_consulta DATE NOT NULL,

horario TIME NOT NULL,

```
status VARCHAR(30),

FOREIGN KEY (id_psicologo) REFERENCES Psicologo(id_psicologo),

FOREIGN KEY (id_paciente) REFERENCES Paciente(id_paciente)

);

CREATE TABLE Avaliacao (

id_avaliacao INT AUTO_INCREMENT PRIMARY KEY,

id_consulta INT NOT NULL,

nota INT,

comentario TEXT,

FOREIGN KEY (id_consulta) REFERENCES Consulta(id_consulta)

);

CREATE TABLE Mensagem (

id_mensagem INT AUTO_INCREMENT PRIMARY KEY,

id_paciente INT NOT NULL,

id_psicologo INT NOT NULL,

texto TEXT,

data_envio DATETIME DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY (id_paciente) REFERENCES Paciente(id_paciente),

FOREIGN KEY (id_psicologo) REFERENCES Psicologo(id_psicologo)

);

CREATE TABLE Disponibilidade (

id_disponibilidade INT AUTO_INCREMENT PRIMARY KEY,

id_psicologo INT NOT NULL,

dia_semana VARCHAR(20),

hora_inicio TIME,

hora_fim TIME,
```

```
FOREIGN KEY (id_psicologo) REFERENCES Psicologo(id_psicologo)  
);
```

# MODELAGEM DE DADOS — NoSQL

## Código

usuarios

```
|
|
|— _id
|— nome
|— email
|— senha
|— telefone
|— tipo_usuario
|— data_cadastro
|— paciente
|  |— data_nascimento
|  |
|  |— psicologo
|  |
|  |— crp
|  |
|  |— valor_consulta
|  |
|  |— descricao
|  |
|  |— especialidades[]
|  |
|  |— disponibilidade[]
```

consultas

```
|
|
|— _id
|— paciente
|— psicologo
```

|— data\_consulta

|— horario

|— status

└— avaliacao

conversas

|

|— \_id

|— paciente

|— psicologo

└— mensagens

O sistema foi desenvolvido utilizando um banco de dados NoSQL orientado a documentos, permitindo maior flexibilidade no armazenamento das informações dos usuários, consultas e conversas. A estrutura foi organizada em três coleções principais: usuários, consultas e conversas.

## Usuários

A coleção de usuários centraliza os dados de autenticação e identificação dos usuários da plataforma. Como o sistema possui dois tipos de usuários (pacientes e psicólogos), foi adotada uma estrutura que permite armazenar informações específicas de cada perfil dentro do mesmo documento.

Os principais atributos armazenados são:

- \* \_id: identificador único do usuário;
- \* nome: nome completo do usuário;
- \* email: endereço de e-mail utilizado para acesso ao sistema;
- \* senha: senha criptografada para autenticação;
- \* telefone: número de contato;
- \* tipo\_usuario: define se o usuário é paciente ou psicólogo;
- \* data\_cadastro: data de criação da conta.

Para usuários do tipo paciente, existe um subdocumento contendo:

- \* data\_nascimento: data de nascimento do paciente.

Para usuários do tipo psicólogo, são armazenadas informações profissionais em um subdocumento específico:

- \* crp: número de registro profissional;
- \* valor\_consulta: valor cobrado por sessão;
- \* descricao: apresentação profissional;
- \* especialidades: lista das áreas de atuação;
- \* disponibilidade: horários disponíveis para atendimento.

Essa estrutura permite que informações específicas de cada perfil sejam armazenadas sem a necessidade de múltiplas tabelas ou relacionamentos complexos.

## **Consultas**

A coleção de consultas é responsável pelo gerenciamento dos agendamentos realizados entre pacientes e psicólogos.

Os campos armazenados são:

- \* \_id: identificador único da consulta;
- \* paciente: referência ao paciente responsável pelo agendamento;
- \* psicologo: referência ao psicólogo que realizará o atendimento;
- \* data\_consulta: data da consulta;
- \* horario: horário agendado;
- \* status: situação atual da consulta (agendada, concluída, cancelada, entre outras);
- \* avaliacao: avaliação realizada pelo paciente após o atendimento.

Essa coleção permite controlar todo o ciclo de vida dos atendimentos realizados na plataforma.

## **Conversas**

A coleção de conversas foi criada para armazenar o histórico de comunicação entre pacientes e psicólogos.

Sua estrutura contém:

- \* \_id: identificador único da conversa;
- \* paciente: referência ao paciente participante;
- \* psicologo: referência ao psicólogo participante;
- \* mensagens: lista contendo todas as mensagens trocadas durante a conversa.

A utilização de um array de mensagens possibilita armazenar todo o histórico de comunicação dentro de um único documento, tornando as consultas mais rápidas e simplificando a recuperação dos dados.

## PADRÕES DE PROJETO (GoF)

Durante o desenvolvimento da plataforma SaaS foram utilizados princípios de organização de software baseados em padrões de projeto, buscando melhorar a estrutura do código, facilitar a manutenção e separar as responsabilidades da aplicação.

A aplicação foi desenvolvida utilizando uma arquitetura baseada em camadas, onde o backend desenvolvido em Node.js com Express atua como camada responsável pelo processamento das requisições e comunicação com o banco de dados MySQL.

O projeto utiliza o conceito do padrão **Singleton**, pertencente aos padrões criacionais do GoF, aplicado no gerenciamento de recursos que devem possuir uma única instância durante a execução da aplicação, como configurações e conexões utilizadas pelo sistema. Essa abordagem evita a criação desnecessária de múltiplos objetos e contribui para um melhor controle dos recursos.

Também foi utilizada a separação de responsabilidades entre aplicação e persistência de dados. O backend é responsável pela lógica da aplicação e comunicação com os bancos de dados, enquanto o MySQL mantém o armazenamento das informações estruturadas, como usuários, consultas, avaliações e dados dos profissionais.

Além disso, a utilização de containers Docker contribuiu para a organização do ambiente da aplicação, permitindo a execução independente dos serviços utilizados pelo sistema, como backend, MySQL, MongoDB e Redis.

A aplicação desses conceitos possibilita uma arquitetura mais organizada, facilitando a evolução da plataforma e a manutenção dos componentes do sistema.